MODULE 02 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Runtimes and standards

Who actually starts a container, and why they are interchangeable.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 2 of 35

admin@sthithapragna.com

Kubernetes talks to **one interface**, so the runtime underneath can change without Kubernetes noticing.

admin@sthithapragna.com

From kubelet to a running process

01

kubelet

The node agent. Decides a container should run.

›

02

CRI

A gRPC API. The kubelet is the client.

›

03

High-level runtime

containerd or CRI-O. Pulls images, manages containers.

›

04

OCI runtime

runc, by default. Creates the process from a bundle.

›

05

Linux kernel

Applies namespaces and cgroups.

ON THE EXAM

Two standards sit on this chain: the CRI above the high-level runtime, and the OCI runtime spec below it.

High-level versus low-level runtime

HIGH-LEVEL RUNTIME

containerd, CRI-O

- Implements the CRI for the kubelet
- Pulls and stores images
- Asks CNI plugins to set up Pod networking
- Runs as a long-lived daemon

LOW-LEVEL RUNTIME

runc, crun

- Implements the OCI runtime spec
- Takes an unpacked filesystem bundle
- Creates namespaces and cgroups
- Starts the process, then steps away

THE TIEBREAKER

If it pulls images or speaks CRI, it is high-level. If it only turns a bundle into a process, it is low-level.

The Open Container Initiative: three specifications

SPECIFICATION	WHAT IT STANDARDISES
Runtime spec	How to run an unpacked filesystem bundle. Reference implementation: runc.
Image spec	The image format: a manifest, a config and a set of layers.
Distribution spec	The API for pushing and pulling images to and from registries.

ON THE EXAM

The OCI is a Linux Foundation project, launched in June 2015 by Docker, CoreOS and others. It is not a CNCF project.

The Container Runtime Interface

- **Defined by Kubernetes** (SIG Node), introduced in 2016. A gRPC API between the kubelet and a runtime.
- **Two services:** RuntimeService for Pods and containers, ImageService for pulling and listing images.
- **Every node needs a CRI runtime.** Without one, the kubelet cannot start a single Pod.
- **crictl** is the CLI for inspecting any CRI runtime on a node.

> REMEMBER
>
> OCI standardises images and low-level runtimes. CRI standardises how the kubelet talks to a runtime. Do not swap them.

Runtimes you should recognise

NAME	WHAT IT IS
containerd	General-purpose CRI runtime. Came from Docker. CNCF graduated, 2019.
CRI-O	Built only for Kubernetes by Red Hat. Versions track Kubernetes. Graduated 2023.
runc	The low-level OCI reference runtime. Used under both of the above.
crun	A fast low-level OCI runtime written in C.
gVisor (runsc)	A sandboxed OCI runtime with a user-space kernel.
Kata Containers	A sandboxed runtime that runs each Pod in a lightweight VM.

ON THE EXAM

containerd and CRI-O are the high-level choices. The rest sit underneath, as OCI runtimes.

The dockershim story

01
Docker only
Early Kubernetes spoke only to Docker Engine.

02
CRI arrives
2016. Any runtime can plug in. Docker Engine never implemented it.

03
dockershim
A shim inside the kubelet translated CRI to Docker.

04
Deprecated
Kubernetes 1.20 announced the removal.

05
Removed
Kubernetes 1.24, May 2022.

ON THE EXAM

cri-dockerd, maintained by Mirantis and Docker, now plays the shim's role for anyone who still wants Docker Engine.

What the removal did, and did not, change

UNCHANGED

Images and laptops

- Images from docker build run everywhere
- They are OCI images
- Developers keep using Docker locally
- Registries work the same

CHANGED

The node runtime

- Nodes need containerd, CRI-O or cri-dockerd
- The kubelet no longer speaks to Docker
- Node tools using the Docker socket break
- Use crictl on nodes instead

THE TIEBREAKER

Kubernetes removed Docker the runtime, not Docker the image format.

Docker Engine versus containerd

DOCKER ENGINE

A developer platform

- CLI, image builds, Compose, networks
- Aimed at people at a keyboard
- Uses containerd underneath
- Does not implement the CRI itself

CONTAINERD

A runtime daemon

- Pulls images, runs containers
- Aimed at other software
- Built-in CRI plugin for the kubelet
- No image builds of its own

THE TIEBREAKER

Docker Engine uses containerd. containerd does not need Docker.

Currency check: runtime versions

- **containerd 1.x:** Kubernetes 1.35 is the last release that supports it.
- **containerd 2.x** is required to upgrade past 1.35. Kubernetes is now at 1.37.
- **CRI-O** versions match Kubernetes minor versions, so it moves in step.
- Old material may still say Docker Engine is the default Kubernetes runtime. Not since 2022.

> ON THE EXAM
>
> If a question asks for the default or recommended runtime today, think containerd or CRI-O, never dockershim.

Sandboxed runtimes: gVisor versus Kata

GVISOR (RUNSC)

A kernel in user space

- Intercepts the app's system calls
- Host kernel sees far fewer calls
- Lighter than a VM
- Some syscalls not supported

KATA CONTAINERS

A VM per Pod

- Each Pod gets its own guest kernel
- Hardware virtualisation boundary
- Stronger isolation
- More memory and start-up cost

THE TIEBREAKER

Both are OCI runtimes, chosen per Pod. Use them for untrusted or multi-tenant code, not by default.

Choosing a runtime per Pod: RuntimeClass

```
 1 apiVersion: node.k8s.io/v1
 2 kind: RuntimeClass
 3 metadata: {name: sandboxed}
 4 handler: runsc
 5 ---
 6 apiVersion: v1
 7 kind: Pod
 8 metadata: {name: partner-fares}
 9 spec:
10   runtimeClassName: sandboxed
11   containers:
12   - name: fares
13     image: partners/fares:3.2.0
```

4 The handler names a runtime already configured on the nodes.

10 Each Pod opts in by name. Pods without it use the default runtime.

Names that are not CRI runtimes

- **systemd.** The Linux init system. It can manage cgroups, but it does not run containers for the kubelet.
- **Podman.** A daemonless, Docker-compatible CLI for people. Not a CRI runtime.
- **kubelet.** The node agent that calls the runtime. It is a client, not a runtime.
- **Buildah, BuildKit.** Image builders. They produce images; they do not run workloads.

> ON THE EXAM
>
> When asked which is not a CRI runtime, look for the init system, the builder, or the human-facing CLI.

A good image is small, pinned, non-root and free of secrets.

admin@sthithapragna.com

The biggest win: a multi-stage build

SINGLE STAGE

Ships the toolchain

```
1 FROM golang:1.25
2 WORKDIR /src
3 COPY . .
4 RUN go build -o /app .
5 CMD ["/app"]
```

Compiler, shell and package manager all ship to production.

MULTI-STAGE

Ships only the binary

```
1 FROM golang:1.25 AS build
2 WORKDIR /src
3 COPY . .
4 RUN CGO_ENABLED=0 go build -o /app .
5
6 FROM gcr.io/distroless/static-debian12
7 COPY --from=build /app /app
8 USER nonroot
9 CMD ["/app"]
```

The build stage is thrown away. The final image holds one file.

THE TIEBREAKER

Build in one stage, copy only the result into a minimal base. Smaller to pull, far less to attack.

Image practices, and why each matters

PRACTICE	WHY
Multi-stage builds	Build tools never reach the final image.
Minimal base: distroless, slim, scratch	Fewer packages, fewer vulnerabilities, faster pulls.
Pin versions, not latest	Every node runs the same thing; rollbacks mean something.
Run as non-root with USER	A breakout starts with fewer privileges.
Use .dockerignore	Keeps .git, .env and junk out of the build context.
Scan images, publish an SBOM	Know what is inside, and find known vulnerabilities.

ON THE EXAM

If an option says fewer layers is the main way to shrink an image, it is overstating a minor effect.

Why a deleted secret is still in the image

```
1 COPY deploy_key /root/.ssh/id_ed25519
2 RUN git clone git@git.example.com:quayside/fares.git
3 RUN rm /root/.ssh/id_ed25519
```

1 The key is now stored in this layer, permanently.

3 The rm only adds a later layer that hides the file. Anyone can extract layer 1.

Use build secrets that never become a layer, or give the key to the running Pod as a Kubernetes Secret.

THE TAKEAWAY

Layers are additive. Deleting a file in a later layer never removes it from an earlier one.

Building without a Docker daemon

- **BuildKit.** Docker's modern build engine, also usable on its own.
- **Buildah and Podman.** Daemonless builds, often used in CI.
- **Cloud Native Buildpacks.** Source code in, OCI image out, no Dockerfile. CNCF graduated, August 2026.
- **kaniko.** Google archived the original in June 2025. Community forks continue.

> ON THE EXAM
>
> Whatever builds it, the output is an OCI image, so any runtime can run it.

> Quayside is moving from an old cluster that used Docker Engine on its nodes to a new cluster running containerd.
>
> Developers ask whether they must rebuild every image with a different tool, and whether they can keep using Docker on their laptops.

Which statement is accurate?

A Images must be rebuilt with Buildah to run on containerd

B Existing images run unchanged, and Docker stays fine locally

C Each node needs the dockershim reinstalled to run the images

D containerd cannot pull images that were pushed to Docker Hub

◆ PAUSE AND DECIDE

Which statement is accurate?

A Images must be rebuilt with Buildah to run on containerd

Docker-built images are OCI images. No rebuild is needed.

B Existing images run unchanged, and Docker stays fine locally

Only the node runtime changed. Images and laptops are unaffected.

C Each node needs the dockershim reinstalled to run the images

The dockershim was removed in 1.24, and containerd needs no shim.

D containerd cannot pull images that were pushed to Docker Hub

Registries follow the OCI distribution spec. containerd pulls from Docker Hub.

THE SIGNAL

Removing Docker as a runtime changed the nodes, not the images.

> Quayside will let partner ferry operators upload their own fare-calculation plugins, which run as containers in the main cluster.
>
> The code is untrusted. Security wants a kernel exploit in a plugin contained, without slowing Quayside's own workloads.

What should the platform team do?

A Put the plugins in their own Kubernetes namespace

B Give each plugin container a strict CPU and memory limit

C Require every plugin to be built on a distroless base image

D Run plugin Pods with a RuntimeClass for gVisor or Kata

◆ PAUSE AND DECIDE

What should the platform team do?

A Put the plugins in their own Kubernetes namespace

A Kubernetes namespace organises objects. The kernel is still shared.

B Give each plugin container a strict CPU and memory limit

Limits stop resource hogging, not kernel exploits.

C Require every plugin to be built on a distroless base image

Good hygiene, but the attacker writes the code. The kernel is still exposed.

D Run plugin Pods with a RuntimeClass for gVisor or Kata

Puts a sandbox or VM boundary around only the untrusted Pods.

THE SIGNAL

Untrusted code plus a kernel-level threat means a sandboxed runtime, applied per Pod.

> An operations script runs docker ps on every node to list running containers. After the move to containerd, it fails on every node.
>
> The team wants a replacement that lists Pods and containers the way the kubelet sees them, on any CRI runtime.

What should the script use?

A crictl, the command-line tool from the cri-tools project

B Install Docker Engine on each node alongside containerd

C runc list, which shows every container runc has created

D kubectl exec into each Pod and list its own processes

◆ PAUSE AND DECIDE

What should the script use?

A crictl, the command-line tool from the cri-tools project

It speaks the CRI, so it sees Pods and containers as the kubelet does.

B Install Docker Engine on each node alongside containerd

Docker would show its own containers, not the kubelet's.

C runc list, which shows every container runc has created

Too low-level: no Pod view, and tied to one OCI runtime.

D kubectl exec into each Pod and list its own processes

Sees inside one container at a time, not what runs on the node.

THE SIGNAL

A question that says 'on any CRI runtime' wants the tool built for the CRI: crictl.

> Quayside's ticket validator image is 1.1 gigabytes. New nodes take minutes to pull it during scale-out.
>
> A scanner reports hundreds of vulnerabilities, almost all in compilers and shell tools the program never uses at run time.

Which single change best addresses both problems?

A Merge every RUN instruction into one, to cut the layer count

B Set imagePullPolicy to Always so nodes pull it more promptly

C Rebuild multi-stage, copying just the binary to a minimal base

D Tag the image latest so every node pulls the newest version

◆ PAUSE AND DECIDE

Which single change best addresses both problems?

A Merge every RUN instruction into one, to cut the layer count

A small saving. The compilers and shells are still in the image.

B Set imagePullPolicy to Always so nodes pull it more promptly

Adds registry checks. The image is just as large and vulnerable.

C Rebuild multi-stage, copying just the binary to a minimal base

Removes the tools: smaller pulls and far fewer vulnerabilities.

D Tag the image latest so every node pulls the newest version

Makes versions unpredictable and forces a pull on every start.

THE SIGNAL

Vulnerabilities in tools the app never uses means those tools should not be in the image.

Module 2 in five lines

- kubelet **calls the CRI**; containerd or CRI-O answer; **runc** creates the process.
- **OCI**: runtime, image and distribution specs. runc is the reference runtime.
- dockershim was **removed in 1.24**. Docker-built images still run everywhere.
- **gVisor and Kata** sandbox untrusted Pods, chosen with a RuntimeClass.
- Good images are **multi-stage, minimal, pinned, non-root and secret-free**.

We have containers and a runtime on every node. Next: **the cluster that tells them what to do**.

admin@sthithapragna.com